SOF 103 Week 2: Variables and Data Types

> Generated by Review Agent on 2026-07-02 06:34 UTC
> Source: `SOF 103-Chapter 02- Variables and Data Types2024_09.ppt`
> Classification confidence: 80%

---

1. Core Knowledge Outline

中文

- **变量 (Variables)**
  - **定义**: 内存中用于存储数据的命名容器。
  - **核心操作**: 声明 (Declaration) 与赋值 (Assignment)。
  - **命名规则**: 必须以字母、下划线或美元符号开头，不能以数字开头；区分大小写；不能使用保留关键字。
  - **最佳实践**: 使用有意义的、驼峰式或下划线式的命名。
- **数据类型 (Data Types)**
  - **基本/原始类型 (Primitive Types)**:
    - **整数 (Integer)**: 存储没有小数部分的数字 (如 `int`, `long`)。
    - **浮点数 (Floating-point)**: 存储带有小数部分的数字 (如 `float`, `double`)。
    - **布尔值 (Boolean)**: 存储逻辑值 `True` 或 `False`。
    - **字符 (Character)**: 存储单个字符 (如 `char`)。
  - **复合/引用类型 (Composite/Reference Types)**:
    - **字符串 (String)**: 存储文本序列 (由字符组成)。
    - **数组 (Array)**: 存储相同类型元素的集合。
    - **对象 (Object)**: 存储键值对集合 (如 Python 的字典, JavaScript 的对象)。
- **类型系统 (Type Systems)**

    - **静态类型 (Statically Typed)**: 变量类型在编译时确定，之后不能更改 (如 Java, C++)。
    - **动态类型 (Dynamically Typed)**: 变量类型在运行时确定，可以随时更改 (如 Python, JavaScript)。
- **类型转换 (Type Conversion)**
    - **隐式转换 (Implicit)**: 由编译器/解释器自动进行，通常是从小范围类型到大范围类型 (安全转换)。
    - **显式转换 (Explicit)**: 由程序员手动进行，通常是从大范围类型到小范围类型 (可能丢失数据或精度)。

English

- **Variables**
    - **Definition**: Named containers in memory for storing data.
    - **Core Operations**: Declaration and Assignment.
    - **Naming Rules**: Must start with a letter, underscore, or dollar sign; case-sensitive; cannot use reserved keywords.
    - **Best Practices**: Use meaningful, camelCase or snake_case names.
- **Data Types**
    - **Primitive Types**:
        - **Integer**: Stores whole numbers (e.g., `int`, `long`).
        - **Floating-point**: Stores numbers with decimal parts (e.g., `float`, `double`).
        - **Boolean**: Stores logical values `True` or `False`.
        - **Character**: Stores a single character (e.g., `char`).
    - **Composite/Reference Types**:
        - **String**: Stores sequences of text (composed of characters).
        - **Array**: Stores a collection of elements of the same type.
        - **Object**: Stores a collection of key-value pairs (e.g., Python dict, JavaScript object).
- **Type Systems**

    - **Statically Typed**: Variable type is determined at compile-time and cannot change (e.g., Java, C++).
    - **Dynamically Typed**: Variable type is determined at runtime and can change (e.g., Python, JavaScript).
- **Type Conversion**
    - **Implicit**: Done automatically by the compiler/interpreter, usually from a smaller to a larger type (safe).
    - **Explicit**: Done manually by the programmer, usually from a larger to a smaller type (potentially lossy).

---

2. Key Concepts & Glossary

Term	Definition	Memory Anchor / Analogy
变量 (Variable)	一个有名字的存储位置，用于在程序中保存和操作数据。 / A named storage location for holding and manipulating data in a program.	**贴了标签的盒子**。你可以把任何东西（数据）放进去，并通过标签（变量名）找到它。 / A **labeled box**. You can put anything (data) inside and find it by its label (variable name).
数据类型 (Data Type)	一种分类，它告诉计算机变量可以存储哪种数据以及可以对其执行哪些操作。 / A classification that tells the computer what kind of data a variable can hold and what operations can be performed on it.	**容器的形状和材质**。一个“整数”容器只能装整数，而一个“字符串”容器只能装文本。 / The **shape and material of a container**. An "integer" container can only hold whole numbers, while a "string" container can only hold text.
声明 (Declaration)	告诉程序你要创建一个新变量的过程。 / The process of telling the program you want to create a new variable.	**向仓库管理员申请一个新盒子**。你告诉管理员盒子的名字和它能装什么。 / **Requesting a new box from the warehouse manager**. You tell the manager the box's name and what it can hold.
赋值 (Assignment)	将一个值放入一个已声明的变量中的过程。 / The process of putting a value into a declared variable.	**把东西放进盒子里**。你可以随时更换盒子里的东西。 / **Putting an item into the box**. You can replace the item inside at any time.
静态类型 (Static Typing)	一种类型系统，其中变量的类型在编译时是已知的，并且之后不能更改。 / A type system where the type of a variable is known at compile time and cannot be changed later.	**一个带有永久标签的盒子**。一旦你告诉仓库管理员这个盒子是装“书”的，它就永远不能用来装“衣服”。 / A **box with a permanent label**. Once you tell the manager the box is for "books," it can never be used for "clothes."
动态类型 (Dynamic	一种类型系统，其中变量的类型在运行时才被解释，并且可以随时更改。 / A type	**一个没有固定标签的盒子**。你可以今天用它装“书”，明天用它装

Term	Definition	Memory Anchor / Analogy
Typing)	system where the type of a variable is interpreted at runtime and can be changed at any time.	“衣服”，只要你想。 / A **box without a fixed label**. You can use it for "books" today and "clothes" tomorrow, whenever you want.
隐式类型转换 (Implicit Conversion)	编译器或解释器自动执行的类型转换，通常是从较小/较安全的类型到较大/较通用的类型。 / Automatic type conversion performed by the compiler/interpreter, usually from a smaller/safer type to a larger/more general type.	**把小硬币换成大钞票**。银行会自动帮你完成，而且你的钱不会变少。 / **Exchanging small coins for a large bill**. The bank does it automatically, and you don't lose any value.
显式类型转换 (Explicit Conversion)	程序员在代码中手动执行的类型转换，通常是从较大/较通用的类型到较小/较具体的类型。 / Manual type conversion performed by the programmer in the code, usually from a larger/more general type to a smaller/more specific type.	**把大钞票强行换成小硬币**。你需要自己去柜台办理，而且可能会损失一些零头（精度丢失）。 / **Forcibly exchanging a large bill for small coins**. You have to go to the counter yourself, and you might lose some change (precision loss).

3. Critical Takeaways

中文

1. **变量是数据的“代号”**：理解变量是内存中存储数据的容器，其核心是“名”与“值”的绑定。考试中常考变量的声明、赋值和命名规则。
2. **数据类型决定了“能做什么”**：不同的数据类型（如整数、字符串）支持不同的操作。例如，你不能对两个字符串进行数学减法。这是编程中常见的错误来源。
3. **区分静态与动态类型**：这是编程语言的一个根本区别。静态类型（如 Java）在编译时检查类型错误，更安全但更“啰嗦”；动态类型（如 Python）更灵活，但运行时更容易出错。面试中常被问到。
4. **警惕类型转换的“陷阱”**：隐式转换通常是安全的，但显式转换（强制转换）可能导致数据丢失或程序崩溃。例如，将浮点数 `3.14` 转换为整数会变成 `3`，丢失了小

数部分。这是考试和实际开发中的高频考点。

5. **常见误区**：混淆“声明”和“赋值”；认为变量名可以以数字开头；忘记在动态类型语言中检查变量类型就进行操作。

English

1. **Variables are "Code Names" for Data**: Understand that a variable is a container in memory, and its core is the binding of a "name" and a "value". Exams often test declaration, assignment, and naming rules.
2. **Data Types Determine "What You Can Do"**: Different data types (e.g., integer, string) support different operations. For example, you cannot perform mathematical subtraction on two strings. This is a common source of errors in programming.
3. **Distinguish Static vs. Dynamic Typing**: This is a fundamental difference between programming languages. Static typing (e.g., Java) checks type errors at compile time, making it safer but more verbose; dynamic typing (e.g., Python) is more flexible but more prone to runtime errors. This is a common interview question.
4. **Beware of the "Pitfalls" of Type Conversion**: Implicit conversion is usually safe, but explicit conversion (casting) can lead to data loss or program crashes. For example, converting the float `3.14` to an integer results in `3`, losing the decimal part. This is a high-frequency test point.
5. **Common Misconceptions**: Confusing "declaration" with "assignment"; thinking variable names can start with a number; forgetting to check a variable's type before operating on it in dynamically typed languages.

---

4. Detailed Notes (AI-Expanded)

4.1 变量：数据的命名容器 / Variables: Named Containers for Data

中文

变量是编程中最基本的概念之一。你可以把它想象成一个**贴了标签的盒子**。这个盒子放在计算机的内存中，用来存放信息（数据）。标签就是**变量名**，盒子里装的东西就是**变**

量的值。

- **为什么需要变量?**

  1. **存储和复用数据**：你可以把用户输入、计算结果等数据存起来，在程序的其他地方反复使用，而不用重复计算或输入。
  2. **提高代码可读性**：使用有意义的变量名（如 `user_age` 而不是 `a`）能让代码像文章一样易于理解。
  3. **方便修改和维护**：如果你需要改变一个值，比如税率，你只需要修改一处变量赋值，而不是在代码中搜索所有用到这个数字的地方。

- **核心操作：声明与赋值**

  - **声明 (Declaration)**：告诉程序“我要创建一个新盒子了”。在静态类型语言中，你还需要告诉程序这个盒子能装什么类型的东西（例如，`int myNumber;`）。在动态类型语言中，你只需要给盒子起个名字（例如，`my_number =`）。
  - **赋值 (Assignment)**：把具体的东西放进盒子里。使用等号 `=` 操作符。例如，`myNumber = 10;` 或 `my_number = 10`。

- **命名规则 (Naming Rules)**：给变量起名字就像给自己的孩子起名字一样，需要遵守一些“家规”：

  - **允许的字符**：只能包含字母（`a-z`，`A-Z`）、数字（`0-9`）、下划线（`_`）和美元符号（`$`）。
  - **不能以数字开头**：`1stPlace` 是无效的，`firstPlace` 是有效的。
  - **区分大小写**：`Name` 和 `name` 是两个完全不同的变量。
  - **不能是保留关键字**：像 `if`，`else`，`while`，`int`，`class` 这些词已经被语言本身占用了，你不能再用它们做变量名。

English

Variables are one of the most fundamental concepts in programming. Think of them as a **labeled box**. This box sits in your computer's memory and is used to hold information (data). The label is the **variable name**, and what's inside the box is the **variable's value**.

- **Why do we need variables?**

    1. **Store and Reuse Data**: You can store data like user input or calculation results and use them repeatedly in other parts of your program without recalculating or re-entering them.
    2. **Improve Code Readability**: Using meaningful variable names (e.g., `user_age` instead of `a`) makes your code as easy to read as a story.
    3. **Facilitate Modification and Maintenance**: If you need to change a value, like a tax rate, you only need to modify the variable assignment in one place, instead of searching through your entire codebase.

- **Core Operations: Declaration and Assignment**

    - **Declaration**: Telling the program, "I'm going to create a new box." In statically typed languages, you also need to specify what *type* of thing the box can hold (e.g., `int myNumber;`). In dynamically typed languages, you just need to give the box a name (e.g., `my_number =`).
    - **Assignment**: Putting a specific item into the box. This is done using the equals sign `=` operator. For example, `myNumber = 10;` or `my_number = 10`.

- **Naming Rules**: Naming a variable is like naming a child; you have to follow some "family rules":

    - **Allowed Characters**: Can only contain letters (`a-z`, `A-Z`), digits (`0-9`), underscores (`_`), and the dollar sign (`$`).
    - **Cannot Start with a Digit**: `1stPlace` is invalid; `firstPlace` is valid.
    - **Case-Sensitive**: `Name` and `name` are two completely different variables.
    - **Cannot be a Reserved Keyword**: Words like `if`, `else`, `while`, `int`, `class` are already used by the language itself and cannot be used as variable names.

4.2 数据类型：变量的“身份证” / Data Types: The Variable's "ID Card"

中文

数据类型定义了变量可以存储的**数据种类**以及可以对其执行的**操作**。它就像是变量的“身份证”，告诉计算机这个变量到底是什么。

- **基本/原始数据类型 (Primitive Data Types)**：这些是语言内置的最基本的数据类型，是构建更复杂数据的“砖块”。
  - **整数 (Integer)**: 用于存储没有小数部分的数字，如 `-5`，`0`，`100`。在 Java 中，常见的有 `int` (32位) 和 `long` (64位)。
  - **浮点数 (Floating-point)**: 用于存储带有小数部分的数字，如 `3.14`，`-0.5`，`2.0e10`。在 Java 中，常见的有 `float` (32位) 和 `double` (64位，精度更高)。
  - **布尔值 (Boolean)**: 只有两个值：`true` 和 `false`。它通常用于表示“是/否”、“开/关”等逻辑状态，是程序进行决策的基础。
  - **字符 (Character)**: 用于存储单个字符，如 `'A'`，`'b'`，`'9'`，`'!'`。在 Java 中，用 `char` 表示，并用单引号括起来。
- **复合/引用数据类型 (Composite/Reference Data Types)**：这些类型是由基本类型或其他复合类型组合而成的更复杂的数据结构。
  - **字符串 (String)**: 这是最常用的复合类型之一。它是由零个或多个字符组成的序列，用于存储文本。例如 `"Hello, World!"`。在 Java 中，`String` 是一个类，而不是基本类型。
  - **数组 (Array)**: 用于存储一组**相同类型**的数据。例如，一个整数数组可以存储 `[1, 5, 2, 8]`。
  - **对象 (Object)**: 在面向对象编程中，对象是类的实例。它可以包含多个不同类型的属性和方法。例如，一个 `Student` 对象可以有 `name` (字符串), `age` (整数), `studentId` (字符串) 等属性。

English

A data type defines the **kind of data** a variable can hold and the **operations** that can be performed on it. It's like the variable's "ID card," telling the computer exactly what the variable is.

- **Primitive Data Types**: These are the most basic, built-in data types of a language. They are the "bricks" for building more complex data.

- **Integer**: Used to store whole numbers without a decimal part, like `-5`, `0`, `100`. In Java, common ones are `int` (32-bit) and `long` (64-bit).
- **Floating-point**: Used to store numbers with a decimal part, like `3.14`, `-0.